

Mauricio de Souza Coutinho

1.Exercício Proposto 1: Escolha tipos para um sistema acadêmico

Modelagem de ALUNO:

- id: INT AUTO_INCREMENT PRIMARY KEY (Identificador numérico autoincrementável).
- matricula: VARCHAR(20) NOT NULL UNIQUE (Código único do aluno; deve ser VARCHAR para preservar zeros à esquerda e evitar operações matemáticas inadequadas).
- nome: VARCHAR(100) NOT NULL (Texto de tamanho variável para o nome completo).
- e-mail: VARCHAR(150) NOT NULL UNIQUE (Endereço de e-mail único).
- nascimento: DATE (Armazena a data de nascimento sem horários desnecessários).
- coeficiente: DECIMAL(4,2) NOT NULL DEFAULT 0.00 (Precisão numérica exata para médias de notas, permitindo valores de 0.00 a 10.00).
- o bolsista: BOOLEAN NOT NULL DEFAULT FALSE (Indicador binário de estado).
- observacoes: TEXT (Campo opcional para anotações livres).

• Diferenciação de Dados:

- Dado obrigatório: Possui restrição NOT NULL (ex: nome).
- Dado ausente: Representado por NULL quando a informação é aplicável mas desconhecida no momento (ex: observacoes vazias).
- Dado inaplicável: Quando o atributo não faz sentido para o registro (evitado por uma modelagem normalizada adequada).

2.Exercício Proposto 2: Declare a tabela RESERVA com constraints coerentes

SQL

```
CREATE TABLE RESERVA (  
    id_reserva INT AUTO_INCREMENT PRIMARY KEY,  
    id_sala INT NOT NULL,      id_responsavel INT  
    NOT NULL,      inicio DATETIME NOT NULL,      fim  
    DATETIME NOT NULL,  
    situacao VARCHAR(20) NOT NULL DEFAULT 'pendente',  
    CONSTRAINT fk_reserva_sala FOREIGN KEY (id_sala) REFERENCES  
    SALA(id_sala),  
    CONSTRAINT fk_reserva_responsavel FOREIGN KEY (id_responsavel)
```

```
REFERENCES USUARIO(id_usuario),
    CONSTRAINT chk_periodo CHECK (fim > inicio),
    CONSTRAINT chk_situacao CHECK (situacao IN ('pendente',
'confirmada', 'cancelada', 'concluida')) );
```

Por que UNIQUE(id_sala, inicio) não detecta toda sobreposição? Essa restrição impede apenas que duas reservas comecem *exatamente no mesmo segundo*. Ela falha se a Reserva A ocorrer das 10:00 às 12:00 e a Reserva B ocorrer das 10:30 às 11:30, pois os horários de início são diferentes, embora os intervalos colidam frontalmente.

3.Exercício Proposto 3: Escolha ON DELETE e ON UPDATE para três vínculos

DEPARTAMENTO -> EMPREGADO (O empregado pode permanecer sem departamento?):

Escolha: ON DELETE SET NULL e ON UPDATE CASCADE.

Justificativa: Se um departamento for extinto, os empregados não devem ser demitidos em cascata; eles ficam temporariamente sem alocação (NULL) até serem remanejados.

Projeto -> TAREFA (A tarefa possui existência independente?):

Escolha: ON DELETE CASCADE e ON UPDATE CASCADE.

Justificativa: Uma tarefa existe exclusivamente no contexto do seu projeto. Se o projeto for excluído, suas tarefas perdem a razão de existir e devem ser removidas automaticamente.

AUTOR -> LIVRO_AUTOR (O contrato histórico pode ser apagado?):

Escolha: ON DELETE RESTRICT (ou NO ACTION) e ON UPDATE CASCADE.

Justificativa: Por razões legais, fiscais e de direitos autorais, o vínculo histórico de autoria não pode ser simplesmente apagado só porque o cadastro do autor foi alterado ou removido.

4.Exercício Proposto 4: Projetos de violações de constraints

Violação de PRIMARY KEY: Inserir duas vezes o mesmo ID em uma tabela

(INSERT INTO ALUNO (id, matricula, ...) VALUES (1, 'A01', ...) executado duas vezes gera erro de chave duplicada).

Violação de UNIQUE e CHECK: Inserir um e-mail já existente (viola UNIQUE) ou tentar cadastrar uma data de término anterior ao início na reserva (fim < inicio, viola o CHECK).

FK Órfã e RESTRICT: Inserir um registro com um id_sala inexistente (ex: 9999) gera erro de chave estrangeira. Tentar excluir um departamento que ainda possui empregados vinculados usando ON DELETE RESTRICT bloqueará a exclusão imediatamente.

Exclusão por CASCADE: Excluir um projeto removerá em cadeia todas as tarefas associadas a ele na tabela dependente.

5.Exercício Proposto 5: Redundâncias e anomalias em relação não normalizada

Análise de VENDA(id_venda, data, id_cliente, nome_cliente, id_produto, nome_produto, quantidade):

- *Fatos misturados:* Dados de clientes, dados de produtos, cabeçalho da venda e itens da venda estão na mesma tabela.
- *Anomalias:*
 - *Atualização:* Mudar o nome de um produto exige alterar todas as linhas onde ele aparece, gerando inconsistências se alguma linha for esquecida.
 - *Inserção:* Impossível cadastrar um novo cliente ou produto sem que haja uma venda associada.
 - *Exclusão:* Apagar a única venda de um cliente apaga permanentemente o cadastro e o nome daquele cliente do sistema. o *Decomposição ideal:* CLIENTE, PRODUTO, VENDA (cabeçalho) e ITEM_VENDA (tabela associativa contendo id_venda, id_produto, quantidade e o **preço unitário histórico** do produto no momento da venda).

6.Desafio Integrador: Implementação do Modelo Físico da Clínica

SQL

```
CREATE DATABASE IF NOT EXISTS clinica_medica;  
USE clinica_medica;
```

```
CREATE TABLE PACIENTE (  
    id_paciente INT AUTO_INCREMENT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,      cpf VARCHAR(14)  
    NOT NULL UNIQUE,      nascimento DATE NOT NULL  
);
```

```
CREATE TABLE MEDICO (  
    id_medico INT AUTO_INCREMENT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,      crm  
    VARCHAR(20) NOT NULL UNIQUE,      especialidade  
    VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE PROCEDIMENTO (  
    id_procedimento INT AUTO_INCREMENT PRIMARY KEY,  
    descricao VARCHAR(100) NOT NULL,  
    valor DECIMAL(10,2) NOT NULL CHECK (valor >= 0) );
```

```
CREATE TABLE CONSULTA (  
    id_consulta INT AUTO_INCREMENT PRIMARY KEY,  
    id_paciente INT NOT NULL,      id_medico INT NOT  
    NULL,      data_hora DATETIME NOT NULL,
```

```

        status VARCHAR(20) NOT NULL DEFAULT 'agendada',
        CONSTRAINT fk_consulta_paciente FOREIGN KEY (id_paciente)
            REFERENCES PACIENTE(id_paciente) ON DELETE RESTRICT ON UPDATE
        CASCADE,
        CONSTRAINT fk_consulta_medico FOREIGN KEY (id_medico)
            REFERENCES MEDICO(id_medico) ON DELETE RESTRICT ON UPDATE
        CASCADE,
        CONSTRAINT chk_status_consulta CHECK (status IN ('agendada',
        'realizada', 'cancelada'))
    );

CREATE TABLE CONSULTA_PROCEDIMENTO (
    id_consulta INT NOT NULL,      id_procedimento
    INT NOT NULL,
    quantidade INT NOT NULL DEFAULT 1 CHECK (quantidade > 0),
    PRIMARY KEY (id_consulta, id_procedimento),
    CONSTRAINT fk_cp_consulta FOREIGN KEY (id_consulta)
        REFERENCES CONSULTA(id_consulta) ON DELETE CASCADE ON UPDATE
    CASCADE,
    CONSTRAINT fk_cp_procedimento FOREIGN KEY (id_procedimento)
        REFERENCES PROCEDIMENTO(id_procedimento) ON DELETE RESTRICT ON
    UPDATE CASCADE );

```

Planejamento de Testes de Violação:

1. **Chave Primária Duplicada:** Inserir um paciente com um `id_paciente` já existente → *Resultado esperado:* Erro de chave primária duplicada.
2. **Chave Estrangeira Órfã:** Inserir uma consulta com `id_paciente = 9999` (inexistente) → *Resultado esperado:* Erro de violação de restrição de chave estrangeira (FK).
3. **CHECK de Valor Negativo:** Inserir um procedimento com preço -50.00 → *Resultado esperado:* Rejeição pelo `CHECK (valor >= 0)`.
4. **CHECK de Status Inválido:** Inserir uma consulta com status 'atendendo' → *Resultado esperado:* Rejeição pelo `CHECK` de vocabulário restrito.
5. **Exclusão Bloqueada (RESTRICT):** Tentar excluir um médico que possui consultas cadastradas → *Resultado esperado:* Operação bloqueada para preservar o histórico médico.
6. **Exclusão em Cascata (CASCADE):** Excluir uma consulta → *Resultado esperado:* Os registros correspondentes na tabela associativa `CONSULTA_PROCEDIMENTO` são apagados automaticamente.